Polony Review

תדריך אישי לקראת ההצגה בהאקתון – מה בנינו, באילו טכנולוגיות, מה הוכח ומה עדיין פתוח

נכתב על בסיס הענף `claude/polony-hitl-app` (הענף המעודכן ביותר, 10.10.2026) בריפו `oriamos/polony-counting` · כולל קריאת כל מסמכי `docs/app`

---

1. הפיץ' ב-30 שניות

הבעיה: במעבדת Lindell (הטכניון) סופרים וירוסים ימיים בשיטת **Polony**. את הנקודות הפלואורסצנטיות (אדום / ירוק / צהוב) על סריקות הג'ל סופרים ידנית – איטי ותלוי באדם.

מה בנינו: Polony Review – אפליקציית ווב מקומית (`FastAPI + React`) בגישת **human-in-the-loop**: שני גלאים עצמאיים (`Big-FISH` קלאסי ו-`Spotiflow` מבוסס AI) מציעים מיקומים, קונצנזוס ביניהם מסמן היכן הם חלוקים, **אדם בודק, מתקן ומאשר**, והמערכת מייצאת ספירות בפורמט המעבדה יחד עם **עקבות מלאים (provenance)**.

הגישה: כל תוצאה אוטומטית היא **הצעה**; אובייקט לא ודאי אף פעם לא נספר; האדם מאשר; כל ייצוא מסומן `DRAFT` או `FINAL` . לא "קופסה שחורה שסופרת" אלא כלי שמזרז ומתעד עבודה מדעית.

2. הרקע והבעיה

- **Polony** = שיטת `solid-phase single-molecule PCR` (Mitra & Church, 1999), שהותאמה ב-2018 (Baran ועמיתים, מעבדת Lindell) לכמת וירוסים ימיים מקלאד מסוים בדגימות שדה.
- ניתוח 20–30 דגימות לוקח כ-2.5 ימים: יומיים במעבדה, ואז **ספירה**. לכל סלייד נסרקים ערוץ אדום (`Cy5`) וערוץ ירוק (`Cy3`); כל נקודה = אמפליקון בודד של פאג; נקודה משותפת לשני הערוצים נראית צהובה.
- הספירה ידנית ומתבצעת ב"שדות" (ריבועי גריד); מהממוצע לשדה מחשבים ריכוז פאגים במ"ל.
- המטרה: להפחית זמן ספירה ידנית, **לסטנדרט** ולהוציא את "הגורם האנושי" – בלי לוותר על שיקול דעת של חוקר.

3. זרימת העבודה במוצר



- **מיפוי ערוצים חובה:** אפליקציה לא מנחשת איזה ערוץ הוא אדום (גם לא לפי שם קובץ או מטא-דאטה); עד שהמשתמש מאשר, זיהוי ואישור חסומים.
- **אזורים:** מלבן/פוליגון לספירה ו"החרגות" (שולי הצ'ימני, רפלקס, פסולת). הסף האוטומטי מחושב רק מהאזור התקף, כך שאזור נכון משפר גם את הזיהוי עצמו.
- **מחלקות אובייקט:** `red` (אדום בלבד), `green` , `yellow` (זוג אדום+ירוק במרחק סובלנות), ו-`uncertain` – **לא נכלל באף סכום**, מדווח בנפרד וחוסם אישור עד שמסווגים או מוחקים.

- **מסך הבדיקה:** זום וגרירה, חמש תצוגות, מגלשי ניגודיות (תצוגה בלבד), קיצורי מקלדת, Inspector לכל אובייקט, פעולות: אישור, הזזה, סיווג מחדש, מיזוג, פיצול, מחיקה רכה, הוספה ידנית.

4. ארכיטקטורה

```
Frontend (React + TS + Vite, Zustand)  <-- REST/JSON -->  FastAPI  (src/polony_app/api)
                                                                |
  store/      SQLite (default)  |  PostgreSQL / Neon (optional, shared)
  detection/  pipeline, masks, matching red<->green, review_reasons, two_engine
  engines/    BigFishAdapter  |  SpotiflowAdapter   (RawDetection לכל מנוע)
  consensus/  Hungarian one-to-one matching  ->  agreement | bigfish_only | spotiflow_only
  review/     state, consensus, counts, geometry, undo/redo, approval fingerprints
  export/     lab.py (XLSX/CSV בפורמט המעבדה) + exporter.py (provenance)
  fields/     grid + reference fields     legacy_import/  legacy data import
  ml/         dataset snapshot, train, evaluate, registry (explicit fine-tuning only)
```

היקף: כ-16.7 אלף שורות `Python` באפליקציה, כ-7.2 אלף שורות `TypeScript` בפרונט (כולל בדיקות), 54 קבצי בדיקות בצד שרת, ו-20 מסמכי תיעוד ב-`docs/app`.

5. הטכנולוגיות שבהן השתמשנו

שכבה	טכנולוגיה	תפקיד
שרת	`Python 3.10+` (נבדק על `3.13`), `FastAPI`, `Uvicorn`, `Pydantic v2`	API, ולידציה, משימות רקע (התקדמות וביטול) לזיהוי
זיהוי קלאסי	`Big-FISH 0.6.2` (LoG), `scikit-image`, `SciPy`, `NumPy`	תיקון רקע, Laplacian-of-Gaussian, סף אוטומטי בנורמלי רעש, מדידת פיק/רקע/SNR
זיהוי AI	`Spotiflow 0.6.5` + `PyTorch` (אופציונלי, `--with-ai`), מודל `general` (88MB~)	רשת heatmap לזיהוי נקודות; רץ גם על CPU (≈21–57 שנ' ל-Hybrid)
קונצנזוס	`SciPy` `linear_sum_assignment` (שיטה הונגרית)	התאמה 1:1 בין שני הגלאים בסובלנות 4 פיקסלים
קריאת תמונות	`tifffile`, `imagecodecs`, `roifile`, `Pillow`	TIFF דו-ערוצי uint16 (בלי המרה ל-8 ביט), קריאת סימוני ImageJ שהחוקרת הטמיעה
אחסון	`SQLite` (מקומי) · `PostgreSQL` / Neon דרך `psycopg 3` + pool	ניתוחים, אובייקטים, undo/redo, לוג פעולות; TIFF נשארים על המחשב ומאומתים ב-`SHA-256`
ייצוא	`openpyxl`	XLSX עם נוסחאות חיות ו-CSV בפורמט המעבדה (T7 ו-Counts)
פרונטאנד	`React 18`, `TypeScript 5.6`, `Vite`, `Zustand 5`	מציג תמונה גדולה עם שכבות, עריכה, אזורים, לוחות סיכום
בדיקות	`pytest` + `httpx`, `vitest`, `Playwright` (e2e מול שרת mock)	552 בדיקות שרת עברו, 32 בדיקות פרונט, 203 בדיקות e2e (לפי המסמכים)
התקנה	סקריפטי `PowerShell / bash`, `uv`	התקנה בפקודה אחת ללא הרשאות מנהל; מוריד Python 3.12 אם צריך; אייקון על שולחן העבודה

CI/CD ותיעוד	`GitHub Actions` (lint, test, install-test, pages, release), `MkDocs`, `semantic-release`	בנוסף: ניסוי היתכנות פריסה ל-`Vercel` (`FastAPI`, Big-FISH על תמונה מלאה ב-1.4 שנ') – מושהה
הקוד הישן	`PyTorch 2.0` (UNet + ResNet18), `h5py`, `wandb`	מסלול הרשת העמוקה של kirill-push; נשמר ללא שינוי ואינו בשימוש האפליקציה (סעיף 10)

6. מנועי הזיהוי והקונצנזוס

Big-FISH (קלאסי, בלי torch)

- תיקון רקע (`gaussian_subtract`, σ=50) ← `LoG` ← סף ← מדידה (פיק, רקע מקומי, SNR, הרוויה, אורך).
- **הסף ביחידות רעש** (תגובת LoG חלקי סיגמא חזק של הרעש) – לכן הזיהוי לא משתנה עם בהירות/חשיפה (נבדק על x0.25, x0.5, x1). סף אוטומטי לפי "מרפק" עקומת הספירה, עם נפילה לערך 5 כשאין אוכלוסיית נקודות.
- `spot_radius_px` (ברירת מחדל 5) הוא סיגמא של ה-LoG – הפרמטר הרגיש ביותר: על התמונה לדוגמה רדיוס 5 ← 108 אובייקטים, 3 ← 207, 2 ← 331.
- פירוק אזורים צפופים (`decompose_dense`) **כבוי**: הוא מפצל פולוניות בהירות לכמה "נקודות" (91 הפכו ל-439).

Spotiflow (AI)

- מודל מאומן מראש עם נרמול `valid_unsaturated` (אחוזונים 1/99.8 של פיקסלים תקפים ולא רוויים) ו-downscale ÷4 לבלוקים גדולים; כל הפרמטרים נרשמים בכל ריצה.
- אם Spotiflow נכשל ב-Hybrid – מתקבלת תוצאת Big-FISH עם אזהרה, ולא תוצאה ריקה בשקט.

קונצנזוס – מה זה אומר (נקודה חשובה בהצגה)

- **agreement**: שני הגלאים ראו נקודה בטווח 4 פיקסלים; **זו לא אמת מוחלטת** (שניהם יכולים להיתפס על אותה פסולת). לכן נשלח לבדיקה מדגם אקראי (`QC sample`, ברירת מחדל 40 לשקופית). נמדד: 15% מה-agreements לא תאמו סימון של החוקרת.
- **bigfish_only / spotiflow_only**: מה שהאדם צריך לראות **קודם** (פסולת בלבד של Big-FISH מקבלת עדיפות בתור). אי-הסכמות הן 24%–85% מהמועמדים, תלוי בשקופית.
- ציוני המנועים אינם הסתברויות מכוילות ולא ניתנים להשוואה זה לזה; בממשק הם נקראים "LoG response" ו-"model score".

7. מה נמדד – תוצאות על 8 שקופיות אמיתיות

> **כנות מדעית:** ההשוואה היא מול סימוני החוקרת (מסמן אחד) – לא מול אמת מוחלטת; הסימונים אינם ממצים, ולכן "דיוק" הוא גבול תחתון. אין כיוון סף על כל 8 השקופיות. מדובר בתוצאות **ראשוניות**.

שקופית	סימונים	Big-FISH רגיש (P / R)	Spotiflow ÷4 סף 0.2 (P / R)
AI-A	332	`0.86 / 0.75`	`0.88 / 0.81`
AI-B	400	`0.81 / 0.77`	`0.83 / 0.79`
FS-A	182	`0.80 / 0.81`	`0.85 / 0.82`
FS-B_D	82	`0.56 / 0.84`	`0.61 / 0.84`

AM-A	100	`0.46 / 0.77`	`0.70 / 0.64`
AM-B	103	`0.44 / 0.54`	`0.60 / 0.51`
AS-A	82	`0.45 / 0.83`	`0.69 / 0.67`
AS-B	60	`0.27 / 0.70`	`0.29 / 0.88`

- **שני משטרים:** שקופיות צפופות (AI, FS) – כל המנועים מסכימים עם הסימונים (דיוק `0.8–0.9`). שקופיות דלילות (AM, AS) – Big-FISH סופר יותר מדי (דיוק `0.25–0.50`), Spotiflow מדויק יותר אבל מפספס חלק.
- Spotiflow לבדו, 8 שקופיות מאוחדות: דיוק/כיסוי **`0.80/0.67`** בסף ברירת המחדל, **`0.73/0.77`** בסף 0.2, **`0.70/0.79`** בסף 0.1. אין סף אחד שמתאים לכל השקופיות.
- בהשוואת ספירה לשדה: שקופיות צפופות `r = 0.82–0.94`; שקופיות דלילות חלשות (`r = 0.04–0.49`) כי הספירה לשדה קטנה (0–8) והאפליקציה סופרת 21–29% יותר.
- הערוץ הירוק כמעט ריק ברוב השקופיות; ההצעות הירוקות של Big-FISH הן בעיקר פסולת. **ירוק וצהוב לא נבדקו מול אמת.**
- **המסקנה שמצדיקה את העיצוב:** אין הגדרה אחת מנצחת ← ולכן תור בדיקה שמתעדף אי-הסכמות, כיול לכל שקופית, ובעתיד fine-tuning על נתונים שאושרו.

8. כללי אמינות ו-provenance

- **טביעת אצבע לאישור:** אישור אזור = `SHA-256` של המצולע, ההחרגות, האובייקטים, תפקידי הערוצים והיישור. כל שינוי מאוחר מבטל אוטומטית את האישור (כולל undo שמחזיר לאותו מצב – אז האישור חוזר).
- **Finalize** דורש: לפחות אזור ספירה אחד, כל האזורים מאושרים, מיפוי ערוצים מאושר וזיהוי עדכני (לא "stale"). אחרי Finalize כל פעולת עריכה מחזירה 409; ניתן `Reopen`.
- **Stale detection:** נגזר מחתימת הבקשה, האזורים והערוצים, לא רק דגל; פרמטרים שהשתנו אחרי הזיהוי מסמנים את התוצאה כמיושנת.
- **לוג פעולות ו-undo/redo** נשמרים וגם שורדים הפעלה מחדש; מחיקות רכות.
- **Provenance:** לכל ריצה – פרמטרים, גרסאות, `checksum` של המודל והמכשיר, hash של קבצי המקור; הכול מיוצא יחד עם הספירות.
- **Pixel size לא מניחים לעולם** (הקובץ אומר px/cm 1000 אך לא אושר) – כל המרחקים בפיקסלים.

9. ייצוא בפורמט המעבדה והנוסחה

- מיוצא רק מניתוחים **מסוימים (Finalized)**. שני סגנונות: טבלת **T7** (שורה לשקופית, `field 1…24`) ו-**Counts** (זוגות A/B, NTC, יעילות). כותרות זהות לאקסל של המעבדה, כולל הטעויות ההיסטוריות בכותרות.
- **הנוסחה (של המעבדה):** `abundance = mean(fields) × (field / grid) / sample_volume × 1000` – כאן `field/grid = 2.9/0.1 = 29`. הממוצע מתעלם מתאים ריקים וכולל אפסים (**ריק = לא נספר**, **0 = נספר, אין**).
- הערך בתא הריכוז נשאר **ריק** עד שכל הקלטים מאושרים (`concentration_formula_confirmed`) – האפליקציה מעדיפה לא להמציא מספר.

10. למה לא המשכנו עם הרשת העמוקה הישנה (שאלה בטוחה שתעלה)

בוצע audit (`LEGACY_AUDIT.md`) – כל חשד נבדק בקוד ובהרצה עם mock. ממצאים מרכזיים של הצינור הישן (`UNet + ResNet18 → CSV`):

ממצא	פירוט
הצינור לא רץ מקצה לקצה	ספירות הן מילונים אבל הייצוא מצפה למספרים; 28 ריבועים (0–27) לעומת קריאה של 1–24 בלבד; חיזוי על קובץ בודד וה-CLI נשברים; ארגומנט נתיב המודל מתעלמים ממנו.
המסווג בפועל לא בשימוש	הסינון בקוד מוערה – ריבועים "ריקים" נספרים בכל זאת.
גריד קבוע וקשיח	158 / 316 פיקסל, מניח תמונה של 2450×1438.
הנחות לא מתועדות	נפח 5µl מקודד קשיח (קבצי המעבדה: 2.5µl – מכפיל 2 בריכוז); T7c לא ממומש; חלוקה ב-100 וקבועים לא מוסברים; `read_tiff` מחלק ב-255 גם ל-16 ביט.
אין אימות	הבדיקות הישנות עושות mock כמעט לכל דבר; לא נמדד מול אמת על שקופיות אמיתיות.

לכן האפליקציה החדשה **לא מייבאת torch או קוד ישן** בבסיס שלה, והקוד הישן נשאר בריפו ללא שינוי, כולל ה-checkpoints (`unet_49_1.7496.pth`, `classifier_57_0.8896.pth`). את המערך הישן אפשר לייבא כנתוני אימון (`legacy_import`).

11. למידה מתוקנים – מה קיים ומה לא

- צינור `ml` מפורש בלבד (האפליקציה לעולם לא מאמנת או מחליפה מודל לבד): `build` (snapshot בלתי-ניתן-לשינוי מנקודות שאדם אישר) ← `train` (fine-tuning של Spotiflow) ← `eval-model` על סט בדיקה נעול ← `promote / rollback`.
- **תוצאה כנה:** ניסוי leave-specimen-out לא הראה שיתרון ל-fine-tuning על המודל המאומן (F1 בטווח −0.03 … +0.02); אימון על 2–3 שקופיות היה לא יציב. תיקנו שני באגים (סקאלת האימון ונרמול), אך כל טענה על שיפור דורשת הערכת leave-one-slide-out.

12. מה פתוח – מוכן לשאלות

פגמים ידועים בצד התוכנה

1. **ה-registry של מודלים לא מחובר לאפליקציה:** מודל "מקודם" לא היה נטען בפועל (`resolve_model` נקרא רק מתוך `polony_app.ml`).
2. סט הבדיקה הנעול לא נאכף בעת קידום מודל (`lock_sha256`, הפרדה בין קבוצות).
3. הדלפות זהות: מקורות נתונים צריכים להיות מפתחים לפי hash תוכן.
4. פער ב-provenance: `source_page` / `source_file` / יישור לא נרשמים תמיד.
5. סקריפט ה-installer ממתין לסטטוס שגוי (`done`); המתקינים של Windows/macOS **לא הורצו אף פעם ב-CI**.

החלטות שממתינות לאנשים

- **Elena (החוקרת):** אמנת הצהוב (בלעדי מול כלול בערוץ); משמעות T7A/T7B (כנראה Clade A/B ולא צבעים); נוסחת הריכוז, דילול ויעילות; מספור התאים ואילו תקפים; נפח 5 מול 2.5µl; יחידות `field/grid` (כנראה ס"מ², לא פיקסלים); משמעות עמודת `QC`.
- **בעל הפרויקט:** גישת GPU ומצב batch (RunPod – לא התחיל); כמה שקופיות מתויגות; האם להפעיל GitHub Actions בפורק; **ריפו ציבורי מול TIFF מחקר וגיליונות מעבדה שנמצאים כרגע במעקב ב-git** (`tests/test_data/lab_templates/`, ובענף `data/research-samples-2026-09-29` יש 8 קבצי TIFF דחוסים) – להחליט אם להסיר; **לסובב את סיסמת ה-Neon**.

13. רשימת משימות להאקתון

לפני האירוע (כשעה)

- ☐ Clone חדש, הפעלת ה-installer על מחשב הדמו, הפעלת האפליקציה ופתיחת שקופית אחת. `--with-ai` רק אם מציגים Spotiflow.
- ☐ הורדה מראש של מודל Spotiflow (`polony-review prepare-model`) – כדי לא להסתמך על אינטרנט על הבמה.
- ☐ בחירת שתי שקופיות לדמו (**אחת צפופה ואחת דלילה**) ושמירת פרויקט גיבוי.
- ☐ הרצה אחת של כל הבדיקות: `make app-test` ו-`npm test` בתיקיית `frontend`.

במהלך ההאקתון – לפי ערך

1. תיקון חסימות 1–2 (חיבור ה-registry ואכיפת סט נעול) – הופך את לולאת הלמידה לאמיתית.
2. כיול סף לכל שקופית מ-30–40 נקודות שנבדקו (ללא אימון).
3. מדגם נבדק של "false positives" כדי למדוד דיוק אמיתי מול סימוני החוקרת.
4. ניסוי fine-tuning: אימון על 6 שקופיות, מבחן על 2 חדשות, קידום רק אם מנצח בסט הנעול.
5. פערי provenance ופריטים קטנים; ואופציונלית מצב RunPod/batch אם יש GPU.

14. הדמו (5 דקות) ומה להגיד בכנות

1. העלאת שקופית ← מיפוי ערוצים (להדגיש: "המערכת לא מנחשת צבע").
2. ציור אזור ספירה והחרגה ← Preview ← הרצת **Hybrid**.
3. שכבת agreement מול disagreement; בדיקת כמה אי-הסכמות (פסולת של Big-FISH בראש התור); undo/redo.
4. אישור אזור ← Finalize ← ייצוא קובץ בפורמט המעבדה + provenance.
5. השוואה לסימוני החוקרת (טבלת סעיף 7).

> **המסר:** "דיוק חלקי ומשתנה בין שקופיות (סעיף 7), האדם נשאר בלולאה, אובייקט לא ודאי לא נספר, ושום אימון לא קורה אוטומטית – ולכן אפשר לסמוך על המספר הסופי ועל מקורו."

15. שאלות צפויות ותשובות

למה human-in-the-loop ולא אוטומציה מלאה?

הדיוק חלקי וקשור לצפיפות השקופית; סימוני החוקרת אינם אמת מוחלטת. עדיף כלי שמזרז בדיקה ומתעד, מאשר מספר סופי לא מבוקר. כל אישור קשור לטביעת אצבע, כך שאי אפשר לשנות בשקט אחרי אישור.

למה שני גלאים?

הם נכשלים אחרת. Big-FISH רגיש אך מפיק פסולת בשקופיות דלילות; Spotiflow מדויק יותר אך מפספס. אי-הסכמה היא האות שאומר לאדם איפה להסתכל קודם.

האם הסכמה בין הגלאים = אמת?

לא. שניהם רואים אותה תמונה ויכולים להיתפס על אותה פסולת (15% מההסכמות לא תאמו סימון). לכן נבדק מדגם אקראי גם מתוכן.

מה הדיוק?

בשקופיות צפופות 0.8–0.9 מול סימוני החוקרת; בדלילות 0.25–0.7 תלוי בגלאי. זה גבול תחתון, מסמן אחד, ואין כיוון סף על כל הסט – ולכן לא טוענים לדיוק מאומת.

למה לא להמשיך עם הרשת העמוקה שכבר הייתה?

הצינור הישן לא רץ מקצה לקצה, הסינון שלו מבוטל, הגריד קשיח, ולא נמדד מול אמת (סעיף 10). הנתונים והמשקולות נשמרו ואפשר להשתמש בהם לאימון.

למה לא מחילים pixel size?

הערך בקובץ (1000 px/cm) לא אושר על ידי החוקרת; האפליקציה עובדת בפיקסלים ולא ממירה עד אישור.

איך מתמודדים עם אי-ודאות בריכוז?

הריכוז המדעי נשאר ריק עד שהחוקרת מאשרת נוסחה, גאומטריה ונפח. הספירות הגולמיות מיוצאות תמיד, עם סימון DRAFT/FINAL.

האם זה שיתופי? מה לגבי נתונים רגישים?

ברירת מחדל מקומית (SQLite, האפליקציה מאזינה ל-127.0.0.1 בלבד, בלי login). אופציה למסד Postgres משותף; קבצי TIFF לא עולים אליו – רק ה-SHA-256 שלהם, וניתוח שנפתח במחשב אחר דורש לצרף את הקובץ המקורי.

מה הצעד הבא?

חיבור registry, כיול לכל שקופית, מדגם נבדק למדידת דיוק אמיתי, ניסוי fine-tuning מבוקר, ואז החלטות החוקרת על הנוסחה.

16. הרצה מהירה

```
# one-command install (Windows PowerShell)
irm https://raw.githubusercontent.com/oriamos/polony-counting/claude/polony-hitl-
app/install/install.ps1 | iex
# macOS / Linux
curl -fsSL https://raw.githubusercontent.com/oriamos/polony-counting/claude/polony-hitl-
app/install/install.sh | bash

# or from a clone
git clone https://github.com/oriamos/polony-counting && cd polony-counting
git checkout claude/polony-hitl-app
make app              # venv + deps + frontend build + server on http://127.0.0.1:8000
```

```
make app-test         # backend tests
polony-review prepare-model   # pre-download the Spotiflow model
```

מסמכים להמשך קריאה: `docs/app/README.md` (התחלה), `USER_GUIDE.md`, `PARAMETERS.md`, `OPEN_ITEMS_AND_HACKATHON.md`, `SCIENTIFIC_OPEN_QUESTIONS.md`, `LEGACY_AUDIT.md`.

> **מה אימתתי בעצמי בסשן הזה:** הרצתי את בדיקות השרת של האפליקציה בסביבה נקייה: 533 עברו, 89 דולגו ו-15 נכשלו (14 מהן בבדיקות ייצוא המעבדה, שמעדכנות XML של קובץ XLSX, ואחת בבדיקת Hybrid שדורשת את חבילת Spotiflow). לא בדקתי אם הכישלונות נובעים מהסביבה שלי, ולכן כדאי להריץ `make app-test` על מחשב הדמו. את האפליקציה עצמה לא הפעלתי, והמספרים בסעיף 7 לקוחים מהמסמכים ולא חושבו מחדש.